

Arquitectura de Módulos y Backlog Base

Proyecto de arquitectura de software — Documento de referencia para el equipo

Objetivo del documento. Establecer una base común y consistente entre el KATA, la base de datos y la arquitectura de microservicios. Este documento define los límites de los módulos y las Historias de Usuario (HUs) de los cuatro sprints. No incluye todavía criterios de aceptación, tareas, subtareas ni asignación de personas.

1. Principio de organización

La arquitectura se organiza alrededor de responsabilidades de negocio. Cada microservicio que posee datos es responsable de su propia información y su propia base de datos. No se deben crear claves foráneas SQL entre bases de datos de distintos servicios; cuando un servicio necesita referenciar una entidad externa, conserva su identificador como referencia lógica y la integridad se resuelve mediante comunicación entre servicios.

Regla clave: microservicio no significa una tabla por servicio. Un servicio puede tener varias tablas relacionadas porque todas pertenecen al mismo contexto de negocio.

2. Mapa general de módulos

El sistema se plantea con seis microservicios principales dueños de datos, más dos módulos transversales y una pieza de infraestructura.

#	Módulo	Responsabilidad principal	Base de datos
1	Auth / Identity	Usuarios, autenticación, sesiones e identidad del tenant	AUTH DB
2	Catalog	Tipos, marcas y reglas de evaluación	CATALOG DB
3	Quotation	Cotizaciones y reglas de precios	QUOTATION DB
4	Orders	Órdenes, cajas, tracking e historial de estados	ORDERS DB
5	Inspection	Inspecciones y resultados del equipo recibido	INSPECTION DB
6	Payment	Intentos, estados y pagos al cliente	PAYMENT DB
7	Notification	Comunicación de cambios de estado y eventos	Sin BD propia obligatoria en MVP
8	Reporting	Consultas y reportes del negocio	Read model / simplificado en MVP

3. Descripción de los módulos para diagramas C4

3.1 Auth / Identity

Responsabilidad: gestionar usuarios, autenticación, sesiones y el contexto de identidad del tenant.

Tablas propietarias: users, sessions

Debe ser el punto responsable de saber quién es el usuario y a qué tenant pertenece. No debe asumir la responsabilidad de cotizaciones, órdenes o pagos.

3.2 Catalog

Responsabilidad: definir qué equipos maneja el sistema y cómo se configuran para ser evaluados.

Tablas propietarias: device_types, device_brands, evaluation_rules

Los tipos y marcas representan el catálogo. Las reglas de evaluación contienen checklists y criterios flexibles en JSONB. La inactivación permite conservar historial sin borrar físicamente información.

3.3 Quotation

Responsabilidad: calcular, registrar y gestionar las cotizaciones.

Tablas propietarias: quotes, pricing_rules

Es el dueño del cálculo de precios. Puede recibir información del catálogo, pero no debe convertirse en dueño del catálogo. La cotización conserva información histórica y puede existir una cotización ajustada después de una inspección.

3.4 Orders

Responsabilidad: administrar el ciclo de vida operativo de una orden después de que una cotización es aceptada.

Tablas propietarias: orders, order_status_history, box_requests

Controla la orden, solicitudes de cajas, tracking y cambios de estado. Mantiene snapshots de datos declarados del equipo para conservar el contexto histórico de la orden.

3.5 Inspection

Responsabilidad: registrar y resolver la inspección física/técnica del equipo recibido.

Tablas propietarias: inspections, inspection_items

Consume las reglas de evaluación del catálogo como referencia, pero es dueño de los resultados reales de la inspección. Puede determinar si el equipo es apto para reventa o debe reciclarse.

3.6 Payment

Responsabilidad: administrar el pago al cliente y sus intentos.

Tablas propietarias: payments

Permite múltiples intentos. Los intentos fallidos permanecen para auditoría y pueden generar nuevos intentos. Solo puede existir un pago COMPLETED por orden, reforzado mediante una restricción de PostgreSQL.

3.7 Notification

Responsabilidad: comunicar eventos relevantes del proceso al cliente o usuarios.

En el MVP puede ser sencillo y no necesita obligatoriamente una base de datos propia. Puede reaccionar a eventos de órdenes, inspección, cotización y pago.

3.8 Reporting

Responsabilidad: proporcionar consultas y reportes para observar el desempeño del negocio.

No debe convertirse en dueño de las entidades operativas. Consume información de otros servicios mediante consultas, eventos o un modelo de lectura. En el MVP puede mantenerse deliberadamente simple.

4. Infraestructura: API Gateway

El API Gateway/BFF local se considera infraestructura, no un módulo de negocio. Su función es recibir las solicitudes del cliente y dirigirlas hacia los microservicios correspondientes. No debe ser dueño de las tablas de usuarios, cotizaciones, órdenes o pagos.

5. Flujo principal del negocio

Cliente → Auth / Identity → Catalog → Quotation → cliente acepta → Orders → solicitud/envío de caja → recepción → Inspection → resultado (reventa/reciclaje) → posible ajuste de Quotation → aceptación final → Payment → Notification / Reporting.

Responsabilidades fundamentales: Inspection determina la condición real; Quotation determina el precio; Orders controla el proceso de la orden; Payment procesa el pago.

6. Backlog maestro — 4 sprints

Las siguientes HUs constituyen la base funcional acordada. En esta etapa no se incluyen criterios de aceptación, tareas, subtareas, story points ni responsables.

ID	Sprint	Módulo	Historia de usuario	Prioridad
HU-001	1	Auth	Configurar identidad y contexto del tenant	Alta
HU-002	1	Auth	Registrar una cuenta de cliente	Alta
HU-003	1	Auth	Iniciar sesión en la plataforma	Alta
HU-004	1	Quotation	Definir reglas básicas de valoración	Alta
HU-005	1	Quotation	Solicitar una cotización de un equipo	Alta
HU-006	2	Catalog	Gestionar tipos de dispositivos	Alta
HU-007	2	Catalog	Gestionar marcas de dispositivos	Media
HU-008	2	Catalog	Gestionar reglas de evaluación	Media
HU-009	2	Quotation	Consultar una cotización registrada	Alta
HU-010	2	Quotation	Aceptar o rechazar una cotización	Alta
HU-011	2	Orders	Crear una orden de reciclaje	Alta
HU-012	3	Orders	Solicitar una caja para una orden	Media
HU-013	3	Orders	Gestionar envío y tracking de una orden	Media
HU-014	3	Orders	Registrar la recepción del equipo	Alta
HU-015	3	Inspection	Registrar la inspección del equipo	Alta
HU-016	3	Inspection	Determinar el resultado de la inspección	Alta
HU-017	3	Quotation	Generar una cotización ajustada	Alta
HU-018	4	Quotation	Aceptar o rechazar el precio ajustado	Alta
HU-019	4	Payment	Crear y procesar un intento de pago	Alta
HU-020	4	Payment	Gestionar reintentos y estados de pago	Alta
HU-021	4	Notification	Consultar notificaciones y estado del proceso	Media
HU-022	4	Reporting / Tenant	Consultar reportes y configuración del negocio	Media

7. Organización por sprint

Sprint 1 — Identidad + cotización inicial

HUs: HU-001, HU-002, HU-003, HU-004, HU-005

Flujo esperado: registro → inicio de sesión → datos del equipo → cálculo → cotización.

Sprint 2 — Catálogo + aceptación + orden

HUs: HU-006, HU-007, HU-008, HU-009, HU-010, HU-011

Flujo esperado: catálogo → cotización persistente → aceptación/rechazo → creación de orden.

Sprint 3 — Logística + inspección + ajuste

HUs: HU-012, HU-013, HU-014, HU-015, HU-016, HU-017

Flujo esperado: caja → envío → recepción → inspección → resultado → posible ajuste de cotización.

Sprint 4 — Aceptación final + pago + seguimiento

HUs: HU-018, HU-019, HU-020, HU-021, HU-022

Flujo esperado: aceptación final → pago/reintento → notificaciones → reportes/configuración.

8. Relación entre HUs y bases de datos

Base de datos	HUs principales
AUTH DB	HU-001, HU-002, HU-003
CATALOG DB	HU-006, HU-007, HU-008
QUOTATION DB	HU-004, HU-005, HU-009, HU-010, HU-017, HU-018
ORDERS DB	HU-011, HU-012, HU-013, HU-014
INSPECTION DB	HU-015, HU-016
PAYMENT DB	HU-019, HU-020
Notification	HU-021
Reporting / Tenant	HU-022

9. Decisiones de consistencia que deben mantenerse

- Los tipos de dispositivo se inactivan en lugar de eliminarse físicamente, preservando historial.
- Una orden puede tener múltiples solicitudes de caja.
- Orders conserva snapshot de marca, modelo, año y condición declarada.
- Payment permite múltiples intentos; los fallidos permanecen para auditoría y solo puede existir un COMPLETED por orden.
- Quotation y Orders mantienen datos históricos necesarios aunque el catálogo cambie.
- Brand se conserva como texto en quotation/orders porque representa un snapshot histórico; no se fuerza una FK entre bases.
- tenant_id se mantiene como referencia lógica; la validación de identidad/tenant pertenece al contexto de Auth/Identity.
- Las transiciones de estado son responsabilidad del servicio; pueden complementarse con CHECKs de valores permitidos.
- Las reglas de evaluación pueden permanecer en JSONB por su naturaleza variable; los resultados de inspección se normalizan en inspection_items.
- No se crean claves foráneas SQL entre las bases de datos de los microservicios.

10. Fuera del alcance de esta versión del backlog

No se incluyen todavía CA, tareas, subtareas, story points, responsables, detalles de endpoints, contratos REST/eventos, despliegue Docker, configuración de CI/CD ni diagramas C4. Esos elementos se definirán después de validar esta base funcional.